

Design and Analysis of Algorithms

Assignment 4

By Himanshu Sardana

Question 1

Given an integer n , the task is to find the solution to the n -queens problem, where n queens are placed on an $n \times n$ chessboard such that no two queens can attack each other. The N Queen is the problem of placing N chess queens on an $N \times N$ chessboard so that no two queens attack each other.

```
#include <bits/stdc++.h>

using namespace std;

bool isSafe(int board[][10], int row, int col, int n) {
    for (int i = 0; i < col; i++)
        if (board[row][i])
            return false;

    for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (board[i][j])
            return false;

    for (int i = row, j = col; j >= 0 && i < n; i++, j--)
        if (board[i][j])
            return false;

    return true;
}

void printSolution(int board[][10], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            cout << board[i][j] << " ";
        cout << endl;
    }
}

void solveNQUtil(int board[][10], int col, int n) {
    if (col >= n) {
```

```

        printSolution(board, n);
        cout << endl;
        return;
    }

    for (int i = 0; i < n; i++) {
        if (isSafe(board, i, col, n)) {
            board[i][col] = 1;
            solveNQUtil(board, col + 1, n);
            board[i][col] = 0; // backtrack
        }
    }
}

void solveNQ(int n) {
    int board[10][10] = {0};
    solveNQUtil(board, 0, n);
}

int main() {
    int n;
    cout << "Enter the value of n: ";
    cin >> n;
    solveNQ(n);
    return 0;
}

```

Question 2

Given an incomplete Sudoku in the form of matrix `mat[][]` of order 9×9 , the task is to complete the Sudoku.

```

#include <bits/stdc++.h>
using namespace std;

const int N = 9;

bool isSafe(vector<vector<int>>& board, int row, int col, int num) {
    for (int x = 0; x < N; x++) {
        if (board[row][x] == num || board[x][col] == num)
            return false;
    }

    int startRow = row - row % 3;
    int startCol = col - col % 3;

    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            if (board[startRow + i][startCol + j] == num)

```

```

        return false;

    return true;
}

bool solveSudoku(vector<vector<int>>& board) {
    for (int row = 0; row < N; row++) {
        for (int col = 0; col < N; col++) {
            if (board[row][col] == 0) {
                for (int num = 1; num ≤ 9; num++) {
                    if (isSafe(board, row, col, num)) {
                        board[row][col] = num;

                        if (solveSudoku(board))
                            return true;

                        board[row][col] = 0;
                    }
                }
                return false;
            }
        }
    }
    return true;
}

void printBoard(const vector<vector<int>>& board) {
    for (const auto& row : board) {
        for (int num : row)
            cout << num << " ";
        cout << "\n";
    }
}

int main() {
    vector<vector<int>> board = {
        {3, 0, 6, 5, 0, 8, 4, 0, 0},
        {5, 2, 0, 0, 0, 0, 0, 0, 0},
        {0, 8, 7, 0, 0, 0, 0, 3, 1},
        {0, 0, 3, 0, 1, 0, 0, 8, 0},
        {9, 0, 0, 8, 6, 3, 0, 0, 5},
        {0, 5, 0, 0, 9, 0, 6, 0, 0},
        {1, 3, 0, 0, 0, 0, 2, 5, 0},
        {0, 0, 0, 0, 0, 0, 0, 7, 4},
        {0, 0, 5, 2, 0, 6, 3, 0, 0}
    };

    if (solveSudoku(board))
        printBoard(board);
}

```

```

    else
        cout << "No solution exists\n";

    return 0;
}

```

Question 3

Given an undirected graph and a number m , the task is to color the given graph with at most m colors such that no two adjacent vertices of the graph are colored with the same color.

```

#include <bits/stdc++.h>
using namespace std;

bool isSafe(int node, int color, vector<vector<int>>& graph, vector<int>& colors) {
    for (int neighbor = 0; neighbor < graph.size(); neighbor++) {
        if (graph[node][neighbor] == 1 && colors[neighbor] == color)
            return false;
    }
    return true;
}

bool solve(int node, vector<vector<int>>& graph, int m, vector<int>& colors)
{
    int n = graph.size();
    if (node == n)
        return true;

    for (int col = 1; col ≤ m; col++) {
        if (isSafe(node, col, graph, colors)) {
            colors[node] = col;

            if (solve(node + 1, graph, m, colors))
                return true;

            colors[node] = 0;
        }
    }
    return false;
}

bool graphColoring(vector<vector<int>>& graph, int m) {
    int n = graph.size();
    vector<int> colors(n, 0);

    return solve(0, graph, m, colors);
}

```

```
int main() {  
    vector<vector<int>> graph = {  
        {0, 1, 1},  
        {1, 0, 1},  
        {1, 1, 0}  
    };  
  
    int m = 3;  
  
    if (graphColoring(graph, m))  
        cout << "Graph can be colored with " << m << " colors.\n";  
    else  
        cout << "Graph cannot be colored with " << m << " colors.\n";  
  
    return 0;  
}
```